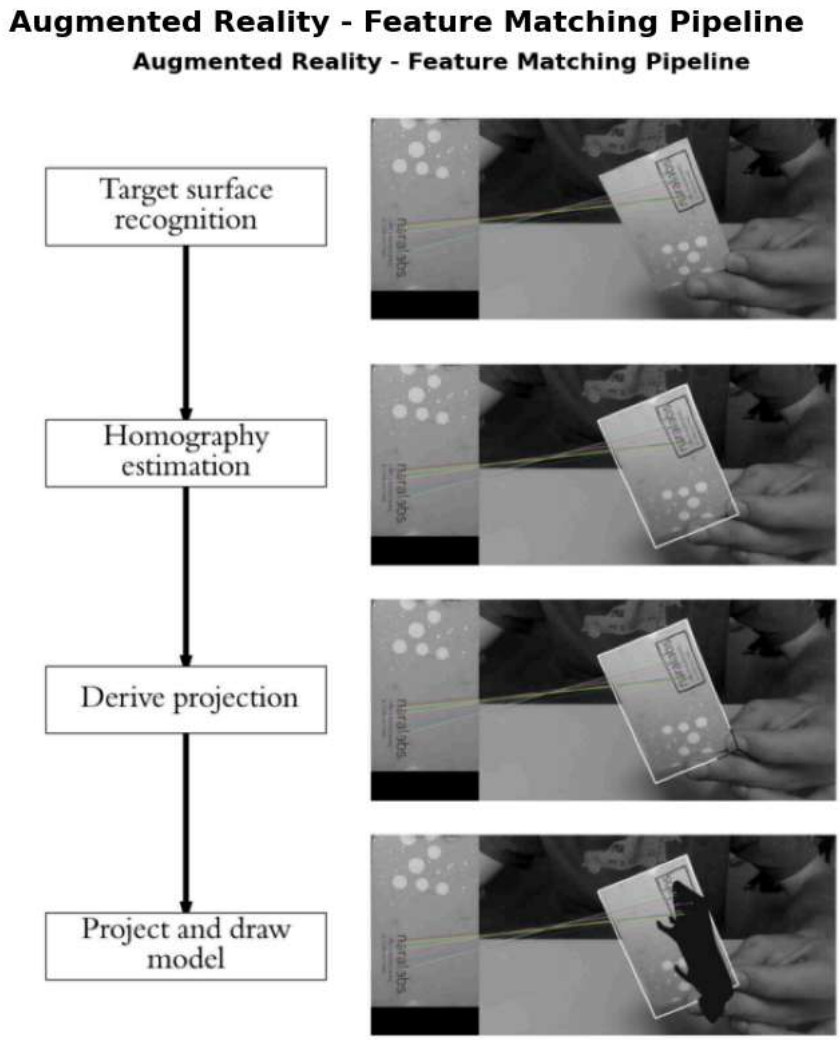
```
import cv2
from matplotlib import pyplot as plt
import os

print("\n" + "="*80)
print("AUGMENTED REALITY - FEATURE MATCHING PIPELINE")
print("="*80 + "\n")

# Use uploaded file name
image_path = "ar-pipe.png"

# Check if file exists
if not os.path.exists(image_path):
    print("Error: ar-pipe.png not found")
    print("Check filename in Colab files panel")
else:
    print("Loading AR Pipeline image...")

    img = cv2.imread(image_path)

    if img is None:
        print("Error: Could not load the image")
    else:
        print("Image loaded successfully!")
        print("Image shape:", img.shape, "\n")

        # Convert BGR → RGB
        img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

        # Display
        plt.figure(figsize=(16, 10))
        plt.imshow(img_rgb)
        plt.title('Augmented Reality - Feature Matching Pipeline',
                  fontsize=16, fontweight='bold')
        plt.axis('off')
        plt.show()

        print("="*80)
        print("AR PIPELINE SUCCESSFULLY DISPLAYED!")
        print("="*80)
```

```
================================================================================
AUGMENTED REALITY - FEATURE MATCHING PIPELINE
================================================================================

Loading AR Pipeline image...
Image loaded successfully!
Image shape: (716, 633, 3)
```




```
================================================================================
AR PIPELINE SUCCESSFULLY DISPLAYED!
================================================================================
```

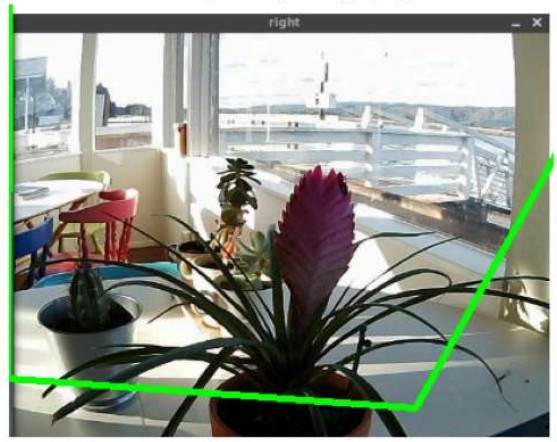
```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# ---------------------------------------------------------
# 1. Load Images
# ---------------------------------------------------------
reference_img = cv2.imread('left.png')
scene_img = cv2.imread('right.png')

if reference_img is None or scene_img is None:
    print("Error: Images not found!")
else:
    # Convert to grayscale
    ref_gray = cv2.cvtColor(reference_img, cv2.COLOR_BGR2GRAY)
    scene_gray = cv2.cvtColor(scene_img, cv2.COLOR_BGR2GRAY)

    # ---------------------------------------------------------
    # 2. Detect Features (ORB)
    # ---------------------------------------------------------
    orb = cv2.ORB_create(nfeatures=1000)
```

```
ref_kp, ref_desc = orb.detectAndCompute(ref_gray, None)
scene_kp, scene_desc = orb.detectAndCompute(scene_gray, None)

# ---------------------------------------------------------
# 3. Match Features
# ---------------------------------------------------------
bf = cv2.BFMatcher(cv2.NORM_HAMMING)

matches = bf.knnMatch(ref_desc, scene_desc, k=2)

good_matches = []
for m, n in matches:
    if m.distance < 0.75 * n.distance:
        good_matches.append(m)

print("Good Matches:", len(good_matches))

# ---------------------------------------------------------
# 4. Compute Homography
# ---------------------------------------------------------
if len(good_matches) > 10:
    src_pts = np.float32([ref_kp[m.queryIdx].pt for m in good_matches]).reshape(-1,1,2)
    dst_pts = np.float32([scene_kp[m.trainIdx].pt for m in good_matches]).reshape(-1,1,2)

    H, mask = cv2.findHomography(src_pts, dst_pts, cv2.RANSAC, 5.0)

    # ---------------------------------------------------------
    # 5. Draw Projection (AR effect)
    # ---------------------------------------------------------
    h, w = ref_gray.shape
    corners = np.float32([[0,0],[w,0],[w,h],[0,h]]).reshape(-1,1,2)

    projected = cv2.perspectiveTransform(corners, H)

    output = scene_img.copy()
    cv2.polylines(output, [np.int32(projected)], True, (0,255,0), 3)

else:
    print("Not enough matches!")
    output = scene_img

# ---------------------------------------------------------
# 6. Draw Matches
# ---------------------------------------------------------
match_img = cv2.drawMatches(reference_img, ref_kp,
                            scene_img, scene_kp,
                            good_matches[:20], None,
                            flags=2)

# ---------------------------------------------------------
# 7. Display Results
# ---------------------------------------------------------
plt.figure(figsize=(15,5))

plt.subplot(1,2,1)
plt.imshow(cv2.cvtColor(match_img, cv2.COLOR_BGR2RGB))
plt.title("Feature Matches")
plt.axis("off")

plt.subplot(1,2,2)
plt.imshow(cv2.cvtColor(output, cv2.COLOR_BGR2RGB))
plt.title("AR Projection (Homography)")
plt.axis("off")

plt.show()
```

```
Good Matches: 55
```

Feature Matches

AR Projection (Homography)



```
# ==========================================================
# AR FEATURE MATCHING PIPELINE (COLAB READY)
# ==========================================================

import cv2
import numpy as np
import matplotlib.pyplot as plt

print("\n" + "="*80)
print("AR FEATURE MATCHING PIPELINE")
print("="*80 + "\n")

# ==========================================================
# STEP 0: LOAD IMAGES
# ==========================================================
print("Loading images...")

reference_img = cv2.imread("left.png")
scene_img = cv2.imread("right.png")

if reference_img is None or scene_img is None:
    print("Error: left.png or right.png not found!")
    exit()

ref_gray = cv2.cvtColor(reference_img, cv2.COLOR_BGR2GRAY)
scene_gray = cv2.cvtColor(scene_img, cv2.COLOR_BGR2GRAY)

ref_h, ref_w = ref_gray.shape

print("Reference Image Shape:", reference_img.shape)
print("Scene Image Shape:", scene_img.shape, "\n")

# ==========================================================
# STEP 1: FEATURE DETECTION (ORB)
# ==========================================================
print("STEP 1: FEATURE DETECTION")

orb = cv2.ORB_create(nfeatures=1000)

ref_kp, ref_desc = orb.detectAndCompute(ref_gray, None)
scene_kp, scene_desc = orb.detectAndCompute(scene_gray, None)

print("Reference keypoints:", len(ref_kp))
print("Scene keypoints:", len(scene_kp), "\n")

# Draw keypoints
ref_kp_img = cv2.drawKeypoints(reference_img, ref_kp, None, color=(0,255,0))
scene_kp_img = cv2.drawKeypoints(scene_img, scene_kp, None, color=(0,255,0))

# Display
plt.figure(figsize=(12,5))
```

```
plt.subplot(1,2,1)
plt.imshow(cv2.cvtColor(ref_kp_img, cv2.COLOR_BGR2RGB))
plt.title("Reference Features")
plt.axis("off")

plt.subplot(1,2,2)
plt.imshow(cv2.cvtColor(scene_kp_img, cv2.COLOR_BGR2RGB))
plt.title("Scene Features")
plt.axis("off")
plt.show()

# ==========================================================
# STEP 2: FEATURE MATCHING
# ==========================================================
print("STEP 2: FEATURE MATCHING")

bf = cv2.BFMatcher(cv2.NORM_HAMMING)

matches = bf.knnMatch(ref_desc, scene_desc, k=2)

good_matches = []
for m, n in matches:
    if m.distance < 0.75 * n.distance:
        good_matches.append(m)

print("Good Matches:", len(good_matches), "\n")

# Draw matches
match_img = cv2.drawMatches(reference_img, ref_kp,
                            scene_img, scene_kp,
                            good_matches[:50], None, flags=2)

plt.figure(figsize=(15,6))
plt.imshow(cv2.cvtColor(match_img, cv2.COLOR_BGR2RGB))
plt.title("Feature Matches")
plt.axis("off")
plt.show()

# ==========================================================
# STEP 3: HOMOGRAPHY
# ==========================================================
print("STEP 3: HOMOGRAPHY")

if len(good_matches) > 10:
    src_pts = np.float32([ref_kp[m.queryIdx].pt for m in good_matches]).reshape(-1,1,2)
    dst_pts = np.float32([scene_kp[m.trainIdx].pt for m in good_matches]).reshape(-1,1,2)

    H, mask = cv2.findHomography(src_pts, dst_pts, cv2.RANSAC, 5.0)

    print("Homography Matrix:\n", H, "\n")

    # Draw detected region
    corners = np.float32([[0,0],[ref_w,0],[ref_w,ref_h],[0,ref_h]]).reshape(-1,1,2)
    projected = cv2.perspectiveTransform(corners, H)

    scene_box = scene_img.copy()
    cv2.polylines(scene_box, [np.int32(projected)], True, (0,255,0), 3)

    plt.imshow(cv2.cvtColor(scene_box, cv2.COLOR_BGR2RGB))
    plt.title("Detected Region")
    plt.axis("off")
    plt.show()

else:
    print("Not enough matches!")
    exit()

# ==========================================================
# STEP 4: DRAW AR OBJECTS (CUBE + AXES)
# ==========================================================
print("STEP 4: AR PROJECTION")

def project_points(H, points):
    projected = []
    for p in points:
        p_homo = np.array([p[0], p[1], 1])
        p_proj = H @ p_homo
```

```
        p_proj = p_proj / p_proj[2]
        projected.append(p_proj[:2])
    return np.array(projected)

# Cube points
size = int(ref_w * 0.5)

cube_3d = np.float32([
    [0,0], [size,0], [size,size], [0,size],
    [0,0], [size,0], [size,size], [0,size]
])

cube_proj = project_points(H, cube_3d)

ar_img = scene_img.copy()

# Draw cube edges
edges = [(0,1),(1,2),(2,3),(3,0),
         (4,5),(5,6),(6,7),(7,4),
         (0,4),(1,5),(2,6),(3,7)]

for e in edges:
    pt1 = tuple(map(int, cube_proj[e[0]]))
    pt2 = tuple(map(int, cube_proj[e[1]]))
    cv2.line(ar_img, pt1, pt2, (255,0,255), 2)

# Draw axes
origin = project_points(H, [[0,0]])[0]
x_axis = project_points(H, [[size,0]])[0]
y_axis = project_points(H, [[0,size]])[0]

cv2.line(ar_img, tuple(origin.astype(int)), tuple(x_axis.astype(int)), (0,0,255), 3)
cv2.line(ar_img, tuple(origin.astype(int)), tuple(y_axis.astype(int)), (0,255,0), 3)

# Show result
plt.figure(figsize=(10,6))
plt.imshow(cv2.cvtColor(ar_img, cv2.COLOR_BGR2RGB))
plt.title("AR Output (Cube + Axes)")
plt.axis("off")
plt.show()

# ========================================================
# DONE
# ========================================================
print("\n" + "="*80)
print("PIPELINE COMPLETE 🚀")
print("="*80)
```

```
================================================================================
AR FEATURE MATCHING PIPELINE
================================================================================

Loading images...
Reference Image Shape: (315, 408, 3)
Scene Image Shape: (325, 405, 3)

STEP 1: FEATURE DETECTION
Reference keypoints: 999
Scene keypoints: 992
```

Reference Features

Scene Features

```
STEP 2: FEATURE MATCHING
Good Matches: 55
```

Feature Matches

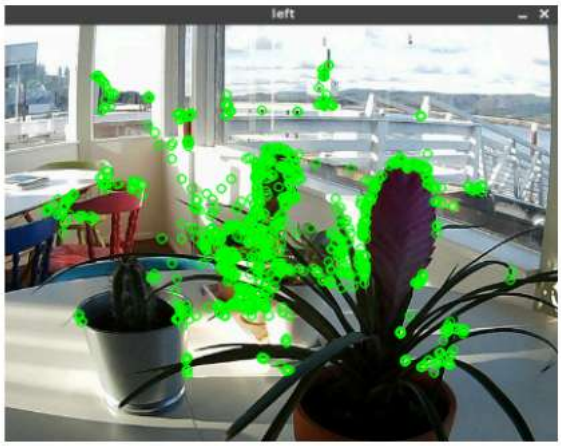
```
# ==========================================================
# AR FEATURE MATCHING PIPELINE (FULL + EXPLANATION)
# ==========================================================

import cv2
import numpy as np
import matplotlib.pyplot as plt

print("\n" + "="*80)
print("AR FEATURE MATCHING PIPELINE")
print("="*80 + "\n")

# ==========================================================
# LOAD IMAGES
# ==========================================================
reference_img = cv2.imread("left.png")
scene_img = cv2.imread("right.png")

if reference_img is None or scene_img is None:
    print("❌ ERROR: Upload left.png and right.png")
    exit()

ref_gray = cv2.cvtColor(reference_img, cv2.COLOR_BGR2GRAY)
scene_gray = cv2.cvtColor(scene_img, cv2.COLOR_BGR2GRAY)

ref_h, ref_w = ref_gray.shape

# ==========================================================
# STEP 1: FEATURE DETECTION
# ==========================================================
print("="*80)
print("STEP 1: TARGET SURFACE RECOGNITION")
print("="*80)
```

```
orb = cv2.ORB_create(nfeatures=1000)

ref_kp, ref_desc = orb.detectAndCompute(ref_gray, None)
scene_kp, scene_desc = orb.detectAndCompute(scene_gray, None)

print(f"Reference features: {len(ref_kp)}")
print(f"Scene features: {len(scene_kp)}\n")

# Show keypoints
ref_kp_img = cv2.drawKeypoints(reference_img, ref_kp, None, color=(0,255,0))
scene_kp_img = cv2.drawKeypoints(scene_img, scene_kp, None, color=(0,255,0))

plt.figure(figsize=(12,5))
plt.subplot(1,2,1)
plt.imshow(cv2.cvtColor(ref_kp_img, cv2.COLOR_BGR2RGB))
plt.title("Reference Features")
plt.axis("off")

plt.subplot(1,2,2)
plt.imshow(cv2.cvtColor(scene_kp_img, cv2.COLOR_BGR2RGB))
plt.title("Scene Features")
plt.axis("off")
plt.show()

# ========================================================
# STEP 2: FEATURE MATCHING
# ========================================================
print("="*80)
print("STEP 2: FEATURE MATCHING")
print("="*80)

bf = cv2.BFMatcher(cv2.NORM_HAMMING)
matches = bf.knnMatch(ref_desc, scene_desc, k=2)

good_matches = []
for m, n in matches:
    if m.distance < 0.75 * n.distance:
        good_matches.append(m)

print(f"Good matches: {len(good_matches)}\n")

# Show matches
match_img = cv2.drawMatches(reference_img, ref_kp,
                            scene_img, scene_kp,
                            good_matches[:50], None, flags=2)

plt.figure(figsize=(15,6))
plt.imshow(cv2.cvtColor(match_img, cv2.COLOR_BGR2RGB))
plt.title("Feature Matches")
plt.axis("off")
plt.show()

# ========================================================
# STEP 3: HOMOGRAPHY
# ========================================================
print("="*80)
print("STEP 3: HOMOGRAPHY")
print("="*80)

if len(good_matches) < 10:
    print("❌ Not enough matches")
    exit()

src_pts = np.float32([ref_kp[m.queryIdx].pt for m in good_matches]).reshape(-1,1,2)
dst_pts = np.float32([scene_kp[m.trainIdx].pt for m in good_matches]).reshape(-1,1,2)

H, mask = cv2.findHomography(src_pts, dst_pts, cv2.RANSAC, 5.0)

print("Homography Matrix:\n", H, "\n")

# Draw detected region
corners = np.float32([[0,0],[ref_w,0],[ref_w,ref_h],[0,ref_h]]).reshape(-1,1,2)
projected = cv2.perspectiveTransform(corners, H)

scene_box = scene_img.copy()
cv2.polylines(scene_box, [np.int32(projected)], True, (0,255,0), 3)
```

```
plt.imshow(cv2.cvtColor(scene_box, cv2.COLOR_BGR2RGB))
plt.title("Detected Marker Region")
plt.axis("off")
plt.show()

# ==========================================================
# STEP 4: AR PROJECTION (CUBE + AXES)
# ==========================================================
print("="*80)
print("STEP 4: AR PROJECTION")
print("="*80)

def project(H, points):
    result = []
    for p in points:
        p_h = np.array([p[0], p[1], 1])
        proj = H @ p_h
        proj = proj / proj[2]
        result.append(proj[:2])
    return np.array(result)

# Define cube
size = int(ref_w * 0.5)

cube = np.float32([
    [0,0],[size,0],[size,size],[0,size],
    [0,0],[size,0],[size,size],[0,size]
])

cube_2d = project(H, cube)

ar_img = scene_img.copy()

# Draw cube
edges = [(0,1),(1,2),(2,3),(3,0),
         (4,5),(5,6),(6,7),(7,4),
         (0,4),(1,5),(2,6),(3,7)]

for e in edges:
    pt1 = tuple(map(int, cube_2d[e[0]]))
    pt2 = tuple(map(int, cube_2d[e[1]]))
    cv2.line(ar_img, pt1, pt2, (255,0,255), 2)

# Draw axes
origin = project(H, [[0,0]])[0]
x_axis = project(H, [[size,0]])[0]
y_axis = project(H, [[0,size]])[0]

cv2.line(ar_img, tuple(origin.astype(int)), tuple(x_axis.astype(int)), (0,0,255), 3)
cv2.line(ar_img, tuple(origin.astype(int)), tuple(y_axis.astype(int)), (0,255,0), 3)

# Show AR output
plt.figure(figsize=(10,6))
plt.imshow(cv2.cvtColor(ar_img, cv2.COLOR_BGR2RGB))
plt.title("AR Output (Cube + Axes)")
plt.axis("off")
plt.show()

# ==========================================================
# FINAL SUMMARY
# ==========================================================
print("\n" + "="*80)
print("PIPELINE COMPLETE 🚀")
print("="*80)

print("""
✓ Step 1: Features detected
✓ Step 2: Features matched
✓ Step 3: Homography computed
✓ Step 4: 3D objects projected

🎯 Result: Augmented Reality Output Generated!
""")
```

```
================================================================================
AR FEATURE MATCHING PIPELINE
================================================================================

================================================================================
STEP 1: TARGET SURFACE RECOGNITION
================================================================================
Reference features: 999
Scene features: 992
```

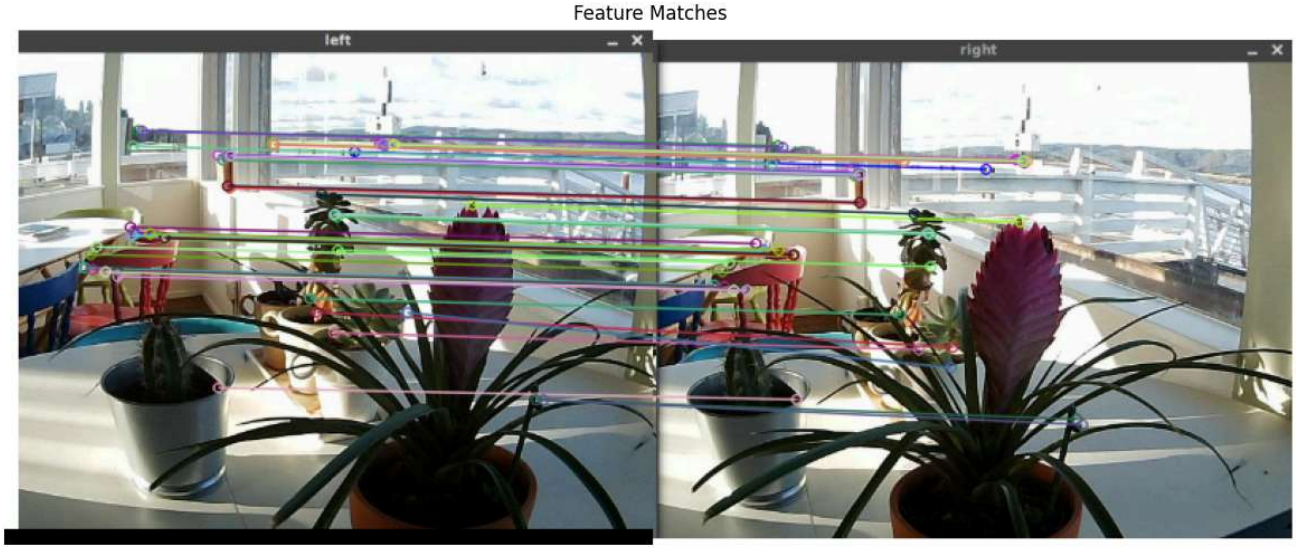
Reference Features

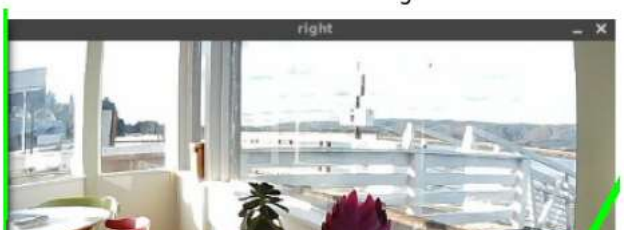


Scene Features

```
================================================================================
STEP 2: FEATURE MATCHING
================================================================================
Good matches: 55
```

Feature Matches




```
================================================================================
STEP 3: HOMOGRAPHY
================================================================================
Homography Matrix:
 [[ 9.86262132e-01 -1.15069703e-02  8.19691217e-01]
 [-3.44036891e-02  1.30695644e+00 -2.31779307e+00]
 [-3.83860815e-04  1.50959525e-03  1.00000000e+00]]
```

Detected Marker Region